Network Port Scanner

Using Python Programming

Design Report

Prepared By
Zoltan Frenyo, El Messaoudi Yasmine &
Vaishnu Balakrishnan Sugumaran

Overview

The **Network & Port Scanner** is a Python-based tool designed to scan hosts on a network and identify which services or ports are exposed. This project combines Python programming, networking concepts, and cybersecurity fundamentals to provide a practical, modular, and user-friendly tool for network analysis.

The project simulates real-world port scanning and allows users to inspect both **console output** and **JSON reports**. Before implementation, the network behavior of port scanning was observed using **Wireshark** to understand how TCP/IP communication occurs and what impact scanning has on network traffic.

Design Report

How your code is structured and why

This Python port scanner project is divided into multiple modules, each responsible for a specific part of the workflow. The main reason for modular design is to organize the code in a clean, logical, and maintainable way. Each module encapsulates a single responsibility:

1. validate_input – Validates IP addresses and port ranges to ensure correctness.
2. socket_connection – Establishes TCP connections to check if a port is open.
3. scan_iterator– Coordinates scanning multiple ports efficiently using concurrency.
4. write_to_console – Formats and displays scan results in the terminal.
5. store_to_file – Saves results to a JSON file for record-keeping.
6. portscanner – Acts as the central controller that orchestrates all modules.

Which libraries you used and what they do

Built-in Libraries

1. **socket**
    - Used to create TCP connections to the target host and ports.
    - Provides functions like socket() and connect_ex() to check if a port is open.
    - Also handles network errors and timeouts.
2. **time**
    - Used to measure the duration of the port scanning process.

- Helps log how long a scan takes.

3. **logging**
    - Provides a flexible logging system for recording the progress of scans.
    - Logs info like scanning start, port status, and scan completion.
    - Helps with debugging and monitoring.
4. **json**
    - Used to save scan results to a JSON file.
    - Converts Python objects (list of dictionaries) into a human-readable JSON format for storage.
5. **typing (List, Tuple)**
    - Provides type hints for function inputs and outputs.
    - Improves code readability and helps with static type checking.
6. **concurrent.futures.ThreadPoolExecutor**
    - Used to scan multiple ports concurrently using threads.
    - Speeds up scanning by performing I/O operations in parallel rather than sequentially.
7. **argparse**
    - It allows a Python script to accept arguments and options from the terminal.
    - Automatically generates help messages and usage instructions.
    - Useful for scripts that need user input when executed from the command line.
8. **re**
    - Helps search, match, split, and replace text patterns.
    - Commonly used for data validation, text parsing, and log processing.
9. **from itertools import repeat**

    repeat () specifically:

    - Repeats the same value multiple times
    - Useful when a function expects an iterable but you want to pass the same constant value
10. **from threading import Lock**
    - Ensures thread-safe operations
    - Prevents race conditions
    - Commonly used when modifying shared variables, files, or APIs.

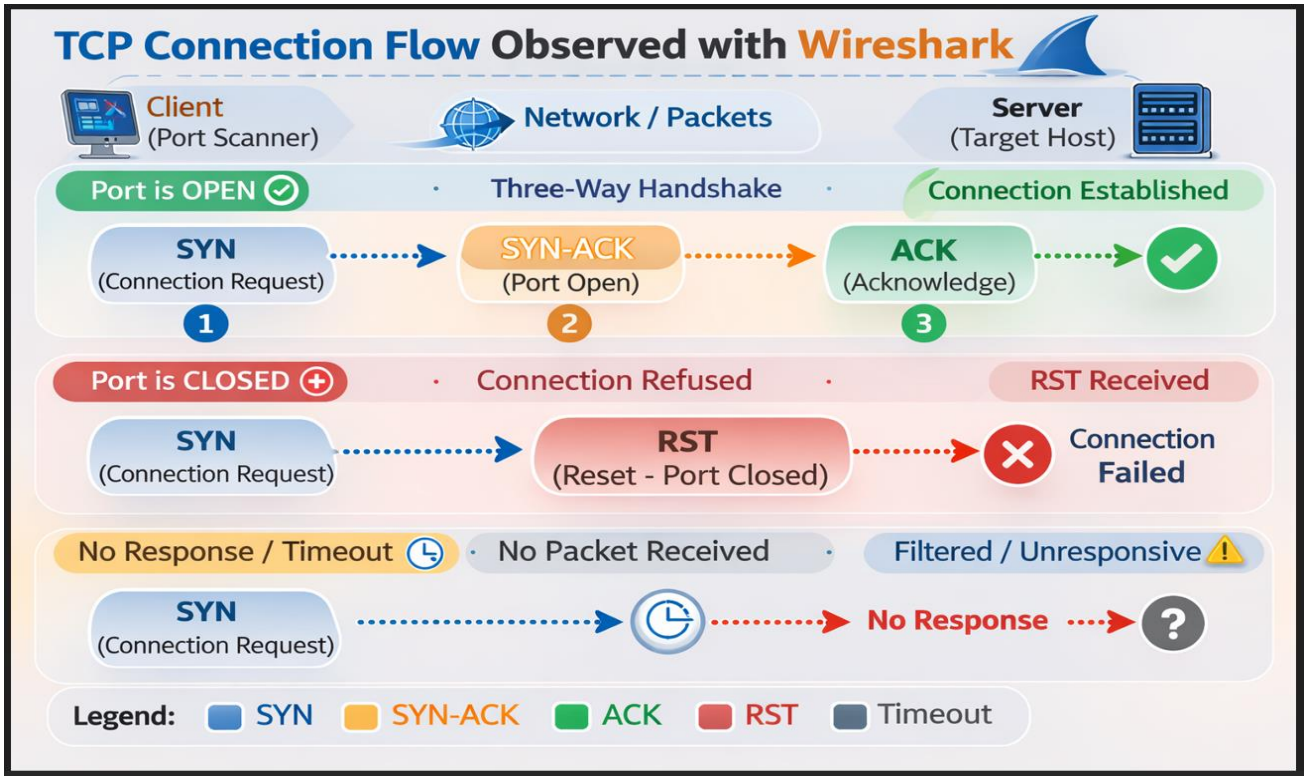
What your Wireshark observations taught you before building

Before implementing the port scanner, we used **Wireshark** to analyse network traffic and observe how TCP connections behave during real port scans. These observations helped shape the design of the project:

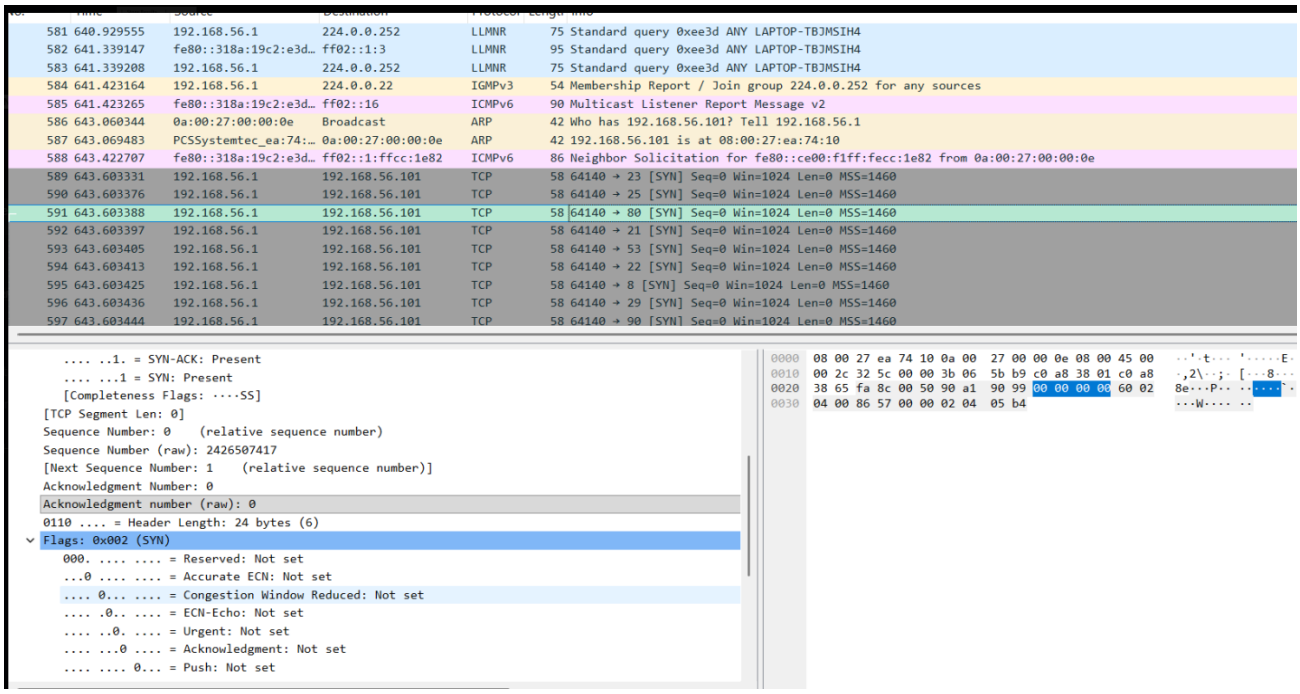
```
Client (Port Scanner) Server (Target Host)
| |
| ----- SYN (connection request) ----> |
| |
| <--- SYN-ACK (port open) ---------- |
| |
| ----- ACK ------------------------> |
| |
| Connection established (port OPEN) |
| |
```

```
If port is CLOSED:

| ----- SYN (connection request) ----> |
| |
| <--- RST (connection refused) ------ |
| |
| Connection failed (port CLOSED) |
```



Port scanner sends a TCP SYN request to the virtual machine to check if a port is open (in this example port 80)

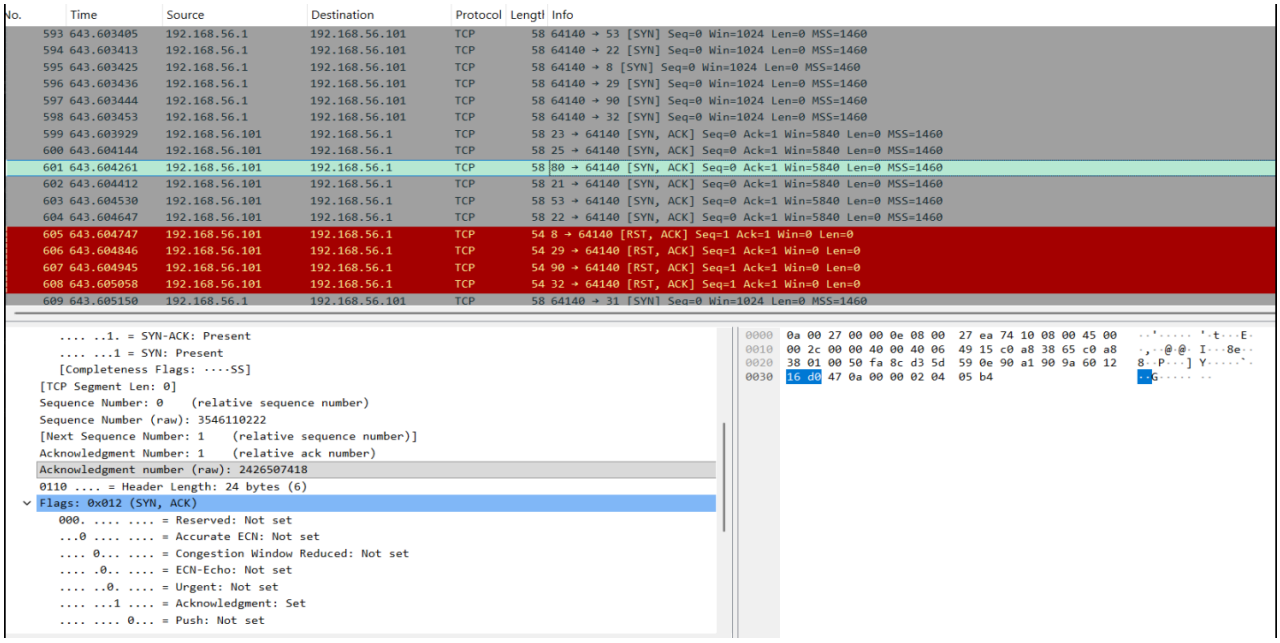


No.	Time	Source	Destination	Protocol	Length	Info
581	640.929555	192.168.56.1	224.0.0.252	LLMNR	75	Standard query 0xee3d ANY LAPTOP-TBJMSIH4
582	641.339147	fe80::318a:19c2:e3d…	ff02::1:3	LLMNR	95	Standard query 0xee3d ANY LAPTOP-TBJMSIH4
583	641.339208	192.168.56.1	224.0.0.252	LLMNR	75	Standard query 0xee3d ANY LAPTOP-TBJMSIH4
584	641.423164	192.168.56.1	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.252 for any sources
585	641.423265	fe80::318a:19c2:e3d…	ff02::16	ICMPv6	90	Multicast Listener Report Message v2
586	643.060344	0a:00:27:00:00:0e	Broadcast	ARP	42	Who has 192.168.56.101? Tell 192.168.56.1
587	643.069483	PCSSystemtec_ea:74:…	0a:00:27:00:00:0e	ARP	42	192.168.56.101 is at 08:00:27:ea:74:10
588	643.422707	fe80::318a:19c2:e3d…	ff02::1:ffcc:1e82	ICMPv6	86	Neighbor Solicitation for fe80::ce00:f1ff:fecc:1e82 from 0a:00:27:00:00:0e
589	643.603331	192.168.56.1	192.168.56.101	TCP	58	64140 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
590	643.603376	192.168.56.1	192.168.56.101	TCP	58	64140 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
591	643.603388	192.168.56.1	192.168.56.101	TCP	58	64140 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
592	643.603397	192.168.56.1	192.168.56.101	TCP	58	64140 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
593	643.603405	192.168.56.1	192.168.56.101	TCP	58	64140 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
594	643.603413	192.168.56.1	192.168.56.101	TCP	58	64140 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
595	643.603425	192.168.56.1	192.168.56.101	TCP	58	64140 → 8 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
596	643.603436	192.168.56.1	192.168.56.101	TCP	58	64140 → 29 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
597	643.603444	192.168.56.1	192.168.56.101	TCP	58	64140 → 90 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

```
    .... ..1. = SYN-ACK: Present
    .... ...1 = SYN: Present
    [Completeness Flags: ····SS]
  [TCP Segment Len: 0]
  Sequence Number: 0    (relative sequence number)
  Sequence Number (raw): 2426507417
  [Next Sequence Number: 1    (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  0110 .... = Header Length: 24 bytes (6)
  Flags: 0x002 (SYN)
    000. .... .... = Reserved: Not set
    ...0 .... .... = Accurate ECN: Not set
    .... 0... .... = Congestion Window Reduced: Not set
    .... .0.. .... = ECN-Echo: Not set
    .... ..0. .... = Urgent: Not set
    .... ...0 .... = Acknowledgment: Not set
    .... .... 0... = Push: Not set
```

```
0000  08 00 27 ea 74 10 0a 00  27 00 00 0e 08 00 45 00   ··'·t··· '·····E·
0010  00 2c 32 5c 00 00 3b 06  5b b9 c0 a8 38 01 c0 a8   ·,2\··;· [···8···
0020  38 65 fa 8c 00 50 90 a1  90 99 00 00 00 00 60 02   8e···P·· ····· ·
0030  04 00 86 57 00 00 02 04  05 b4                     ···W···· ··
```

The virtual machine answers with SYN, ACK => the port is open

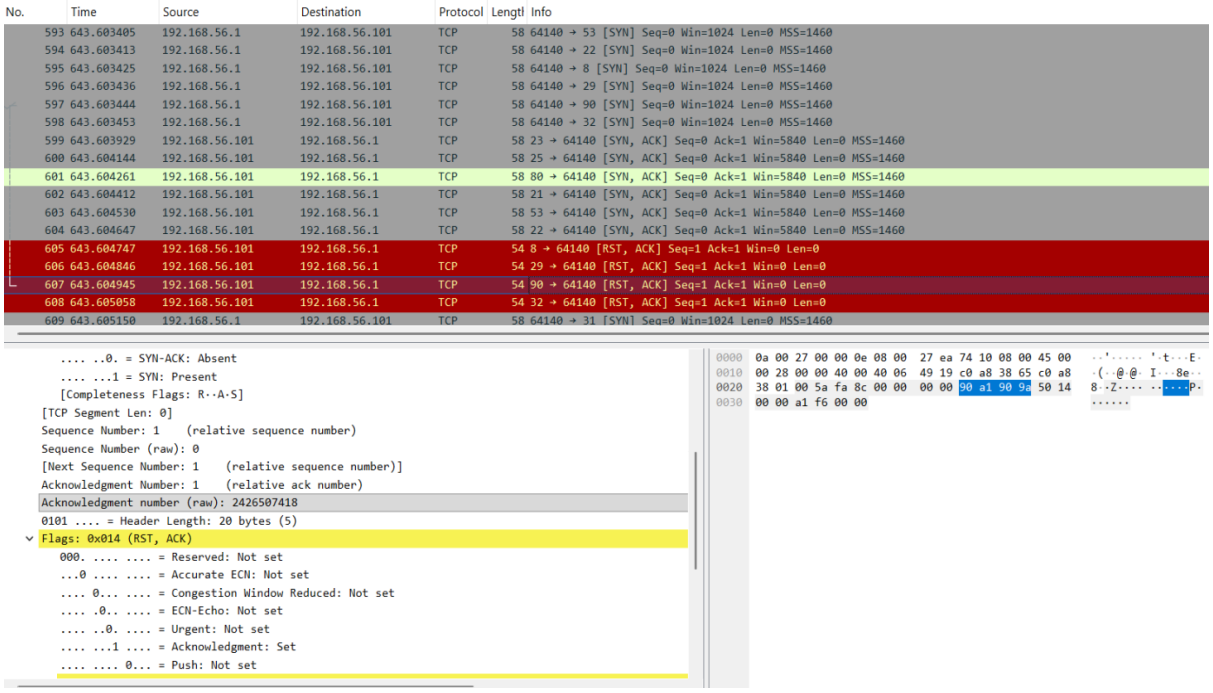


No.	Time	Source	Destination	Protocol	Length	Info
593	643.603405	192.168.56.1	192.168.56.101	TCP	58	64140 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
594	643.603413	192.168.56.1	192.168.56.101	TCP	58	64140 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
595	643.603425	192.168.56.1	192.168.56.101	TCP	58	64140 → 8 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
596	643.603436	192.168.56.1	192.168.56.101	TCP	58	64140 → 29 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
597	643.603444	192.168.56.1	192.168.56.101	TCP	58	64140 → 90 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
598	643.603453	192.168.56.1	192.168.56.101	TCP	58	64140 → 32 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
599	643.603929	192.168.56.101	192.168.56.1	TCP	58	23 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
600	643.604144	192.168.56.101	192.168.56.1	TCP	58	25 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
601	643.604261	192.168.56.101	192.168.56.1	TCP	58	80 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
602	643.604412	192.168.56.101	192.168.56.1	TCP	58	21 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
603	643.604530	192.168.56.101	192.168.56.1	TCP	58	53 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
604	643.604647	192.168.56.101	192.168.56.1	TCP	58	22 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
605	643.604747	192.168.56.101	192.168.56.1	TCP	54	8 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
606	643.604846	192.168.56.101	192.168.56.1	TCP	54	29 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
607	643.604945	192.168.56.101	192.168.56.1	TCP	54	90 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
608	643.605058	192.168.56.101	192.168.56.1	TCP	54	32 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
609	643.605150	192.168.56.1	192.168.56.101	TCP	58	64140 → 31 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

```
.... ..1. = SYN-ACK: Present
.... ...1 = SYN: Present
[Completeness Flags: ····SS]
[TCP Segment Len: 0]
Sequence Number: 0    (relative sequence number)
Sequence Number (raw): 3546110222
[Next Sequence Number: 1    (relative sequence number)]
Acknowledgment Number: 1    (relative ack number)
Acknowledgment number (raw): 2426507418
0110 .... = Header Length: 24 bytes (6)
Flags: 0x012 (SYN, ACK)
  000. .... .... = Reserved: Not set
  ...0 .... .... = Accurate ECN: Not set
  .... 0... .... = Congestion Window Reduced: Not set
  .... .0.. .... = ECN-Echo: Not set
  .... ..0. .... = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 0... = Push: Not set
```

```
0000  0a 00 27 00 00 0e 08 00  27 ea 74 10 08 00 45 00   ··'····· '·t···E·
0010  00 2c 00 00 40 00 40 06  49 15 c0 a8 38 65 c0 a8   ·,··@·@· I···8e··
0020  38 01 00 50 fa 8c d3 5d  59 0e 90 a1 90 9a 60 12   8··P···] Y·····`·
0030  16 d0 47 0a 00 00 02 04  05 b4                     ··G····· ··
```

The virtual machine answers with RST, ACK => the port is closed



No.	Time	Source	Destination	Protocol	Length	Info
593	643.603405	192.168.56.1	192.168.56.101	TCP	58	64140 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
594	643.603413	192.168.56.1	192.168.56.101	TCP	58	64140 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
595	643.603425	192.168.56.1	192.168.56.101	TCP	58	64140 → 8 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
596	643.603436	192.168.56.1	192.168.56.101	TCP	58	64140 → 29 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
597	643.603444	192.168.56.1	192.168.56.101	TCP	58	64140 → 90 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
598	643.603453	192.168.56.1	192.168.56.101	TCP	58	64140 → 32 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
599	643.603929	192.168.56.101	192.168.56.1	TCP	58	23 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
600	643.604144	192.168.56.101	192.168.56.1	TCP	58	25 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
601	643.604261	192.168.56.101	192.168.56.1	TCP	58	80 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
602	643.604412	192.168.56.101	192.168.56.1	TCP	58	21 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
603	643.604530	192.168.56.101	192.168.56.1	TCP	58	53 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
604	643.604647	192.168.56.101	192.168.56.1	TCP	58	22 → 64140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
605	643.604747	192.168.56.101	192.168.56.1	TCP	54	8 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
606	643.604846	192.168.56.101	192.168.56.1	TCP	54	29 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
607	643.604945	192.168.56.101	192.168.56.1	TCP	54	90 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
608	643.605058	192.168.56.101	192.168.56.1	TCP	54	32 → 64140 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
609	643.605150	192.168.56.1	192.168.56.101	TCP	58	64140 → 31 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

```
.... ..0. = SYN-ACK: Absent
.... ...1 = SYN: Present
[Completeness Flags: R··A·S]
[TCP Segment Len: 0]
Sequence Number: 1    (relative sequence number)
Sequence Number (raw): 0
[Next Sequence Number: 1    (relative sequence number)]
Acknowledgment Number: 1    (relative ack number)
Acknowledgment number (raw): 2426507418
0101 .... = Header Length: 20 bytes (5)
Flags: 0x014 (RST, ACK)
  000. .... .... = Reserved: Not set
  ...0 .... .... = Accurate ECN: Not set
  .... 0... .... = Congestion Window Reduced: Not set
  .... .0.. .... = ECN-Echo: Not set
  .... ..0. .... = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 0... = Push: Not set
```

```
0000  0a 00 27 00 00 0e 08 00  27 ea 74 10 08 00 45 00   ··'····· '·t···E·
0010  00 28 00 00 40 00 40 06  49 19 c0 a8 38 65 c0 a8   ·(··@·@· I···8e··
0020  38 01 00 5a fa 8c 00 00  00 00 90 a1 90 9a 50 14   8··Z···· ····P·
0030  00 00 a1 f6 00 00                                  ····· ·
```

1. **TCP Handshake Understanding**
    - Observed the **three-way handshake** (SYN → SYN-ACK → ACK) when connecting to open ports.
    - Noted that **closed ports often respond with RST** (reset), indicating the connection is refused.
    - This confirmed that checking connection success via socket.connect_ex() would reliably indicate open or closed ports.
2. **Timeouts and Unresponsive Hosts**
    - Some ports or hosts did not respond, resulting in **timeouts**.
    - This emphasized the need to **set socket timeouts** in the scanner to prevent it from hanging.
3. **Port Scanning Behavior**
    - Observed sequential scans produce **slow performance**, especially with large port ranges.
    - Reinforced the benefit of **concurrent scanning using threads** to speed up the process.
4. **Common Port Identification**
    - By capturing traffic to common services (HTTP, SSH, DNS), it was easier to **map port numbers to service names**.
    - Helped in implementing the **"common ports" feature** in the console output.
5. **Error Handling**
    - Noted various **network errors**, such as name resolution failures or unreachable hosts.
    - Confirmed the need for **exception handling** to ensure the scanner does not crash.

Modules

validate_input()

↓

port_scanner()

↓

Scan_iterator() (ThreadPoolExecutor)

↓

socket_connection()

↓

write_to_console()

↓

store_to_file()

1. **validate_input module**

This module contains functions that validate user inputs for the port scanner to ensure they are correctly formatted and safe to use. It verifies that a single port number is numeric and within the valid range of 0–65535, and it can also validate a port range in the format start-end, ensuring both values are valid and that the start port is not greater than the end port. The module also checks whether an IPv4 address is correctly structured by confirming it contains four numeric octets separated by periods and that each octet is between 0 and 255. Additionally, it validates the output filename, ensuring it ends with .json and only contains allowed characters such as letters, numbers, underscores, dashes, and periods. If any input fails validation, the functions raise a ValueError with a descriptive error message. A small test section at the bottom allows the module to be run independently to verify that the validation functions behave correctly with sample inputs.

```python
"""
validate_input.py

Contains functions to validate IP addresses, ports, port ranges, and output
filenames.

This module ensures:
- IP addresses are valid IPv4
- Ports are numeric and within 0-65535
- Port ranges are properly formatted (start <= end)
- Output filenames are valid JSON files with allowed characters
"""

from typing import List
import re

# ----------------------------
# Port validation
# ----------------------------
def validate_port(input_port: str) -> List[int]:
    """
    Validate a single port input.

    Args:
        input_port (str): Port as a string (e.g., '80').

    Returns:
        List[int]: List containing a single valid port number.

    Raises:
        ValueError: If the port is invalid.
    """
    input_port = input_port.strip()

    if input_port == "":
        raise ValueError("PORT ERROR: Port number is empty!")

    try:
        port = int(input_port)
    except ValueError:
        if " " in input_port:
            raise ValueError("PORT ERROR: Port cannot contain spaces!")
        else:
            raise ValueError("PORT ERROR: Port must be a number between 0 and
65535!")

    if port < 0:
        raise ValueError("PORT ERROR: Port cannot be negative!")
    elif port > 65535:
        raise ValueError("PORT ERROR: Port cannot be greater than 65535!")

    return [port]


def validate_port_range(input_port: str) -> List[int]:
```

```python
    """
    Validate a range of ports (start-end).

    Args:
        input_port (str): Port range as a string (e.g., '20-25').

    Returns:
        List[int]: List of all port numbers in the range.

    Raises:
        ValueError: If the range is invalid.
    """
    # Split string by dash
    ports = [p.strip() for p in input_port.split("-")]

    if len(ports) != 2:
        raise ValueError("PORT ERROR: Port range must be in 'start-end'
format!")
    if "" in ports:
        raise ValueError("PORT ERROR: Ports cannot be empty!")

    start_port_str, end_port_str = ports

    try:
        start_port = int(start_port_str)
    except ValueError:
        raise ValueError("PORT ERROR: Start port is invalid!")

    try:
        end_port = int(end_port_str)
    except ValueError:
        raise ValueError("PORT ERROR: End port is invalid!")

    if start_port < 0 or end_port < 0:
        raise ValueError("PORT ERROR: Ports cannot be negative!")
    if start_port > 65535 or end_port > 65535:
        raise ValueError("PORT ERROR: Ports cannot be greater than 65535!")
    if start_port > end_port:
        raise ValueError("PORT ERROR: Start port must be <= end port!")

    # Generate list of ports
    return list(range(start_port, end_port + 1))


# ----------------------------
# IP validation
# ----------------------------
def validate_ip(ip_address: str) -> str:
    """
    Validate an IPv4 address.

    Args:
        ip_address (str): IP address as a string.

    Returns:
```

```python
        str: Validated IP address.

    Raises:
        ValueError: If the IP is invalid.
    """
    ip_address = ip_address.strip()

    if " " in ip_address:
        raise ValueError("IP ERROR: Spaces are not allowed in IP address!")

    octets = ip_address.split(".")

    if len(octets) != 4:
        raise ValueError("IP ERROR: IP address must have 4 octets!")

    for o in octets:
        try:
            value = int(o)
        except ValueError:
            raise ValueError("IP ERROR: Each octet must be numeric!")

        if not (0 <= value <= 255):
            raise ValueError("IP ERROR: Octets must be between 0 and 255!")

    return ip_address


# ----------------------------
# Filename validation
# ----------------------------
def validate_filename(filename: str) -> str:
    """
    Validate the output JSON filename.

    Only allows letters, numbers, underscores, dashes, and periods.
    Must end with .json.

    Args:
        filename (str): Filename string.

    Returns:
        str: Validated filename.

    Raises:
        ValueError: If filename is invalid.
    """
    filename = filename.strip()

    if not filename.endswith(".json"):
        raise ValueError("FILE ERROR: Output file must end with .json")

    if not re.match(r'^[A-Za-z0-9_\-\.]+$', filename):
        raise ValueError("FILE ERROR: Filename contains invalid characters")
```

```python
    return filename


# ----------------------------
# Example / test (optional)
# ----------------------------
if __name__ == "__main__":
    # This is just a small test harness; can be removed in production
    test_cases = [
        ("192.168.0.1", "80"),
        ("192.168.0.1", "20-25"),
        ("127.0.0.1", "0-65535"),
        ("256.0.0.1", "80"),   # invalid IP
        ("192.168.0.1", "-1"), # invalid port
    ]

    for ip, ports in test_cases:
        try:
            ip_valid = validate_ip(ip)
            try:
                port_list = validate_port_range(ports)
            except ValueError:
                port_list = validate_port(ports)
            print(f"{ip_valid} -> {port_list}")
        except ValueError as e:
            print(f"Error for {ip}, {ports}: {e}")
```

2. portscanner module

This module is the main entry point for a Python-based port scanner that operates through a command-line interface. It begins by parsing command-line arguments provided by the user, such as the target IP address, the port or range of ports to scan, the number of threads, the maximum connection rate, the timeout value, and the output file name. After reading these inputs, the script validates them to ensure the IP address, ports, and filename are correctly formatted. Once validated, it calls the scan_iterator function to perform the actual port scanning by attempting connections to each specified port on the target machine. The results, indicating whether each port is open or closed, are then displayed in the console using write_to_console and saved to a JSON file using store_to_file. If any input is invalid, the program prints an error message and terminates.

```python
"""
portscanner.py

Main entry point for the Python Port Scanner project.

This script :
- Parses command line arguments (IP, port(s), threads, rate, timeout, output
file)
- Validates the inputs
- Calls the scanning function (scan_iterator)
- Outputs results to the console and to a JSON file

CLI-only mode : Interactive mode was removed because it introduced several
untested edge cases when no CLI arguments were provided. The scanner now runs
exclusively via the command-line interface.
"""

from typing import List
from functions.scan_iterator import scan_iterator
from functions.write_to_console import write_to_console
from functions.store_to_file import store_to_file
from functions.validate_input import validate_ip, validate_port,
validate_port_range, validate_filename
import logging
import argparse

# ----------------------------
# Logging setup
# ----------------------------
logger = logging.getLogger(__name__)
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    force=True,
)

# ----------------------------
# Core function
# ----------------------------
def portscanner(ip_address: str,
                ports: List[int],
                max_threads: int,
                max_connections_per_sec: float,
                output_file: str,
                timeout: float):
    """
    Runs the port scanner on a given IP and list of ports.

    Args:
        ip_address (str): Target IPv4 address.
        ports (List[int]): List of ports to scan.
        max_threads (int): Maximum number of concurrent threads.
        max_connections_per_sec (float): Maximum global connection attempts
per second.
        output_file (str): JSON filename to store results.
```

```python
        timeout (float): Socket timeout in seconds.

    Returns:
        None. Results are written to console and output file.
    """

    # Perform port scanning
    scan_result: List[tuple[int, bool]] = scan_iterator(
        ip_address, ports, max_threads, max_connections_per_sec, timeout
    )

    # Output results to console
    write_to_console(ip_address, scan_result)

    # Store results in JSON file
    store_to_file(ip_address, scan_result, output_file)


# ----------------------------
# Command line interface (CLI)
# ----------------------------
def main():
    """
    Parses CLI arguments and runs the portscanner.
    """

    # Argument parser
    parser = argparse.ArgumentParser(
        description="Python Port Scanner",
        epilog="Example: python3 portscanner.py -t 192.168.56.101 -p 20-100"
    )

    # Required arguments
    parser.add_argument(
        "-t", "--target", type=str, required=True,
        help="Target IP address (example: 192.168.1.10)"
    )
    parser.add_argument(
        "-p", "--ports", type=str, required=True,
        help="Port or range of ports (example: 80 or 20-100)"
    )

    # Optional arguments
    parser.add_argument(
        "-th", "--threads", type=int, default=100,
        help="Maximum number of threads (default = 100)"
    )
    parser.add_argument(
        "-r", "--rate", type=float, default=100.0,
        help="Maximum connections per second (default = 100.0)"
    )
    parser.add_argument(
        "-ti", "--timeout", type=float, default=1.0,
        help="Socket timeout in seconds (default = 1.0)"
    )
```

```python
    parser.add_argument(
        "-o", "--output", default="scan_results.json",
        help="Output JSON file (default: scan_results.json)"
    )

    # Read arguments
    args = parser.parse_args()

    # Validate inputs
    try:
        ip_address = validate_ip(args.target)

        try:
            ports = validate_port_range(args.ports)
        except ValueError:
            ports = validate_port(args.ports)

        output_file = validate_filename(args.output)

    except ValueError as e:
        print(f"Input Error: {e}")
        exit(1)

    # Run the scanner
    portscanner(
        ip_address=ip_address,
        ports=ports,
        max_threads=args.threads,
        max_connections_per_sec=args.rate,
        output_file=output_file,
        timeout=args.timeout
    )


# ----------------------------
# Entry point
# ----------------------------
if __name__ == "__main__":
    main()s
```

3. socket_connection module

This module is responsible for attempting a TCP connection to a single port on a target IP address as part of the port scanning process. The socket_connection function creates a TCP socket using Python's socket library and tries to connect to the specified ip_address and port. A timeout is set to prevent the program from hanging if the target does not respond. The function uses connect_ex, which returns 0 if the connection succeeds, indicating that the port is open, and a non-zero value if the connection fails, indicating that the port is closed or

unreachable. The function then returns a tuple containing the port number and a boolean value showing whether the port is open (True) or closed (False). Logging statements are included for debugging purposes to record the result of each scan. Additionally, the function handles potential errors such as name resolution failures (socket.gaierror) and other network or socket issues (OSError), ensuring that the scan does not crash and instead safely returns the port as closed.

```
"""
socket_connection.py

Handles the actual TCP connection to a single port.

This module:
- Attempts to connect to a target IP:port.
- Uses socket timeouts to avoid hanging.
- Returns a tuple (port, status) where status is True if open, False if
closed.
"""

from typing import Tuple
import socket
import logging

# ----------------------------
# Single port connection
# ----------------------------
def socket_connection(ip_address: str, port: int, timeout: float) ->
Tuple[int, bool]:
    """
    Attempt to connect to a single TCP port on a target IP.

    Args:
        ip_address (str): Target IPv4 address.
        port (int): TCP port to scan.
        timeout (float): Socket timeout in seconds.

    Returns:
        Tuple[int, bool]: (port, True if open, False if closed)

    Notes:
        - Uses context manager `with` to automatically close the socket.
        - Catches name resolution errors and network/socket errors.
    """
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.settimeout(timeout)  # Set socket timeout
            result = sock.connect_ex((ip_address, port))  # 0 if success,
otherwise error code
            is_open = (result == 0)

            # Logging for debugging
```

```
            if is_open:
                logging.debug(f"Finished scanning {ip_address}:{port} - OPEN")
            else:
                logging.debug(f"Finished scanning {ip_address}:{port} -
CLOSED")

            return (port, is_open)

    # Name resolution failed
    except socket.gaierror:
        logging.debug(f"Scan aborted for {ip_address}:{port} - name resolution
failed")
        return (port, False)

    # Other network/socket errors
    except OSError:
        logging.debug(f"Scan aborted for {ip_address}:{port} - network/socket
error")
        return (port, False)
```

4. scan_iterator module

This module is responsible for scanning multiple ports on a target IP address concurrently. It uses Python's ThreadPoolExecutor to perform port scans in parallel, which significantly speeds up the scanning process compared to scanning ports one by one. Before each scan, a thread-safe rate limiter ensures that the number of connection attempts does not exceed a specified limit per second, preventing excessive network requests. The scan_port function waits for permission from the rate limiter and then calls socket_connection to attempt a TCP connection to the specified port. The main function, scan_iterator, receives the target IP, the list of ports to scan, the maximum number of threads, the allowed connection rate, and the socket timeout. It then distributes the scanning tasks across multiple threads and collects the results as a list of tuples containing the port number and a boolean value indicating whether the port is open. Logging messages record the start and completion of the scan along with the total time taken.

```
"""
scan_iterator.py

Responsible for iterating over a list of ports and scanning them concurrently.

Features:
- Uses ThreadPoolExecutor for multithreading.
- Supports rate limiting (max connections per second).
- Returns a list of tuples (port, is_open).
"""
```

```python
from typing import List, Tuple
from .socket_connection import socket_connection
import time
from concurrent.futures import ThreadPoolExecutor
from itertools import repeat
from threading import Lock
import logging

# ----------------------------
# Rate limiter class
# ----------------------------
class RateLimiter:
    """
    Thread-safe rate limiter to control the number of scans per second.

    Attributes:
        interval (float): Minimum time (in seconds) between scan attempts.
        lock (Lock): Ensures only one thread updates next_allowed_time at a
time.
        next_allowed_time (float): Monotonic time for the next allowed scan.

    Example:
        rate_limiter = RateLimiter(rate=10.0)  # 10 scans/sec max
        rate_limiter.wait()  # wait until allowed to scan
    """

    def __init__(self, rate: float) -> None:
        if rate <= 0:
            raise ValueError("Rate must be greater than 0")

        self.interval = 1.0 / rate
        self.lock = Lock()
        self.next_allowed_time = time.monotonic()

    def wait(self) -> None:
        """
        Waits until the next allowed scan time according to rate limit.
        Ensures all threads respect the global rate limit.
        """
        with self.lock:
            now = time.monotonic()
            if now < self.next_allowed_time:
                time.sleep(self.next_allowed_time - now)
            # Schedule next allowed time
            self.next_allowed_time = max(self.next_allowed_time +
self.interval, time.monotonic())

# ----------------------------
# Single port scan
# ----------------------------
def scan_port(ip: str, port: int, limiter: RateLimiter, timeout: float) ->
Tuple[int, bool]:
    """
    Scans a single TCP port on a target IP, respecting rate limits.
```

```python
    Args:
        ip (str): Target IP address.
        port (int): Target port number.
        limiter (RateLimiter): Shared rate limiter instance.
        timeout (float): Socket timeout in seconds.

    Returns:
        Tuple[int, bool]: (port, True if open, False if closed)
    """
    limiter.wait()  # enforce global rate limit
    return socket_connection(ip, port, timeout)


# ----------------------------
# Main scan iterator
# ----------------------------
def scan_iterator(ip_address: str,
                  ports: List[int],
                  max_threads: int,
                  max_connections_per_sec: float,
                  timeout: float) -> List[Tuple[int, bool]]:
    """
    Loops through a list of ports and attempts to connect to each of them on
the specified IP address.

    Args:
        ip_address (str): Target IP address.
        ports (List[int]): List of ports to scan.
        max_threads (int): Maximum concurrent threads.
        max_connections_per_sec (float): Max total connection attempts per
second.
        timeout (float): Socket timeout in seconds.

    Returns:
        List[Tuple[int, bool]]: List of (port, status) tuples where
status=True if port is open.
    """
    if not ports:
        return []

    logging.info(
        f"Starting port scan on {ip_address} ports {ports[0]}-{ports[-1]} "
        f"with max threads: {max_threads}, max connections/sec:
{max_connections_per_sec}"
    )

    start_time = time.time()
    rate_limiter = RateLimiter(max_connections_per_sec)

    # Use ThreadPoolExecutor to scan ports concurrently
    with ThreadPoolExecutor(max_workers=max_threads) as executor:
        # repeat(ip_address) and repeat(timeout) create a value for each port
        # executor.map preserves port order in results
        output_port_list: List[Tuple[int, bool]] = list(
```

```
            executor.map(scan_port, repeat(ip_address), ports,
repeat(rate_limiter), repeat(timeout))
        )

    elapsed_time = time.time() - start_time
    logging.info(f"Port scan finished in {elapsed_time:.5f} seconds")

    return output_port_list
```

5. write-to-console module

This module is responsible for displaying the results of the port scan in a clear and formatted table in the console. It defines a dictionary of common ports and their associated services (for example, port 22 for SSH or port 80 for HTTP) so that when a port is scanned, the program can show the likely service running on that port. The main function, write_to_console, receives the target IP address and a list of scan results, where each result contains a port number and a boolean value indicating whether the port is open or closed. It then prints a structured table showing the IP address, port number, port status (OPEN or CLOSED), and the common service name if the port is recognized. This makes the scan results easier for a user to read and quickly understand which services might be available on the target machine. An example block at the bottom demonstrates how the function works by printing sample scan results when the module is run directly.

```
"""
write_to_console.py

Handles printing port scan results to the console in a formatted table.

Features:
- Shows IP, port, status (OPEN/CLOSED), and common service name if known.
- Simple, human-readable output for quick scanning feedback.
"""

from typing import List, Tuple

# Mapping of common ports to services
COMMON_PORTS = {
    20: "FTP", 21: "FTP", 22: "SSH", 23: "Telnet",
    25: "SMTP", 53: "DNS", 80: "HTTP", 110: "POP3",
    119: "NNTP", 123: "NTP", 143: "IMAP", 161: "SNMP",
    179: "BGP", 443: "HTTPS", 465: "SMTPS", 500: "ISAKMP",
    587: "SMTP", 993: "IMAPS", 995: "POP3S",
    1433: "MSSQL", 1521: "Oracle", 2049: "NFS",
    2082: "cPanel", 2083: "cPanel SSL", 2086: "WHM",
    2087: "WHM SSL", 2181: "Zookeeper", 2222: "DirectAdmin",
```

```python
    2375: "Docker", 2376: "Docker SSL", 2483: "Oracle TCPS",
    2484: "Oracle", 3000: "Dev Server", 3306: "MySQL",
    3389: "RDP", 3690: "Subversion", 4444: "Metasploit",
    4567: "Ruby", 5000: "Flask", 5060: "SIP",
    5432: "PostgreSQL", 5601: "Kibana", 5672: "RabbitMQ",
    5900: "VNC", 5985: "WinRM", 5986: "WinRM SSL",
    6379: "Redis", 6443: "Kubernetes", 6667: "IRC",
    7001: "WebLogic", 7002: "WebLogic SSL",
    7199: "Cassandra", 7474: "Neo4j", 8000: "HTTP Alt",
    8008: "HTTP Proxy", 8009: "AJP13", 8080: "HTTP Proxy",
    8081: "HTTP Alt", 8088: "Hadoop", 8090: "HTTP Alt",
    8161: "ActiveMQ", 8200: "Vault", 8333: "Bitcoin",
    8443: "HTTPS Alt", 8500: "Consul", 8761: "Eureka",
    8888: "Jupyter", 9000: "SonarQube", 9042: "Cassandra",
    9092: "Kafka", 9093: "Kafka SSL", 9200: "Elasticsearch",
    9418: "Git", 9999: "Debug"
}

# ----------------------------
# Console output function
# ----------------------------
def write_to_console(ip_address: str, scan_result: List[Tuple[int, bool]]) ->
None:
    """
    Prints a formatted table of port scan results to the console.

    Args:
        ip_address (str): Target IP address.
        scan_result (List[Tuple[int, bool]]): List of tuples (port, is_open)

    Example:
        write_to_console("192.168.1.1", [(21, True), (22, False)])
    """
    print("\nScan Results")
    print("-" * 60)
    print(f"{'IP':<15}{'PORT':<10}{'STATUS':<10}{'COMMON PORT'}")
    print("-" * 60)

    for port, status in scan_result:
        state = "OPEN" if status else "CLOSED"
        common = COMMON_PORTS.get(port, "No")
        print(f"{ip_address:<15}{port:<10}{state:<10}{common}")


# ----------------------------
# Example usage
# ----------------------------
if __name__ == "__main__":
    ip_address = "192.168.1.1"
    scan_results = [
        (21, True),
        (22, False),
        (23, False),
        (80, True),
        (8761, False),
        (26, True)
```

```
    ]

    write_to_console(ip_address, scan_results)
```

6. **store-to-file module**

This module is responsible for saving the port scan results to a JSON file. The store_to_file function takes the scanned IP address, a list of scan results, and the name of the output file as input. Each scan result is originally stored as a tuple containing a port number and a boolean value indicating whether the port is open or closed. The function converts these tuples into a list of dictionaries so that the data can be properly structured in JSON format. Each dictionary includes the IP address, the port number, and the port status, which is written as "OPEN" or "CLOSED". The data is then written to the specified file using Python's json.dump function with indentation to make the file easier to read. An example block at the bottom of the script demonstrates how the function can be used by saving sample scan results to a JSON file.

```
"""
store_to_file.py

Handles storing port scan results to a JSON file.

Features:
- Converts a list of tuples (port, status) into a JSON list of dictionaries.
- Each dictionary contains: IP address, port number, and status
("OPEN"/"CLOSED").
- Indented JSON for readability.
"""

import json
from typing import List, Tuple

# ----------------------------
# Store scan results to JSON
# ----------------------------
def store_to_file(ip_address: str, scan_result: List[Tuple[int, bool]],
filename: str) -> None:
    """
    Stores port scan results in a JSON file.

    Args:
        ip_address (str): The scanned IP address.
        scan_result (List[Tuple[int, bool]]): List of (port, status) tuples.
        filename (str): Output JSON file name.

    Returns:
        None
```

```
    Example:
        store_to_file("192.168.1.1", [(21, True), (22, False)],
"results.json")
    """
    data = []

    # Convert tuple list to list of dictionaries for JSON
    for port, status in scan_result:
        data.append({
            "ip": ip_address,
            "port": port,
            "status": "OPEN" if status else "CLOSED"
        })

    # Write to file
    with open(filename, "w") as file:
        json.dump(data, file, indent=4)

# ----------------------------
# Example usage
# ----------------------------
if __name__ == "__main__":
    ip_address = "192.168.1.1"
    results = [
        (21, True),
        (22, False),
        (23, False),
        (80, True)
    ]
    store_to_file(ip_address, results, "scan_results.json")
    print("Scan results saved to scan_results.json")
```

SYN scan Vs TCP

The difference between a SYN scan and a TCP scan mainly comes from how they interact with the TCP connection process during port scanning. These are commonly used in network security tools like Nmap.

1. SYN Scan (Half-Open Scan)

A SYN scan is often called a half-open scan because it does not complete the full TCP handshake.

How it works

1. Scanner sends a SYN packet to the target port.
2. If the port is open, the target replies with SYN-ACK.
3. Instead of completing the handshake, the scanner sends RST (reset) to close the connection.

Result logic

- SYN-ACK → Port is OPEN
- RST → Port is CLOSED
- No response / ICMP → Filtered

Key characteristics

- Faster
- Stealthier (connection never fully established)
- Requires root/admin privileges
- Often used in penetration testing

2. TCP Connect Scan (Full TCP Scan)

A TCP scan (usually called TCP Connect Scan) completes the full TCP handshake.

How it works

1. Scanner sends SYN
2. Target replies SYN-ACK
3. Scanner responds with ACK → connection established
4. Connection is then closed.

Result logic

- Connection successful → Port OPEN
- RST → Port CLOSED

Key characteristics

- Less stealthy
- Works without root privileges

- Easier to detect in logs
- Slightly slower

Feature	**SYN Scan**	**TCP Connect Scan**
TCP handshake	Partial (half-open)	Full
Speed	Faster	Slower
Stealth	Stealthier	Easier to detect
Privileges needed	Root/Admin	Normal user
Detection in logs	Less likely	Very likely
TCP handshake	Partial (half-open)	Full